

Threads

Reference:

A Programmer's Guide to Java SCJP Certification - A Comprehensive Primer (3rd Edition).

Khalid A. Mughal and Rolf W. Rasmussen

Addison-Wesley, Boston, ISBN: 0321556054 , 2009.

Chapter 13

Overview

- Multi-tasking
- Threads
- Thread Creation
- Synchronization
- Thread Transitions

Multitasking

- Multitasking allows several activities to occur concurrently on the computer.
- A distinction is usually made between:
 - Process-based multitasking
 - Thread-based multitasking
- Some advantages of thread-based multitasking:
 - Threads share the same address space.
 - Context switching between threads is usually inexpensive.
 - Communication between threads is usually inexpensive.

Threads

- A *thread* is a path of execution within a program, that is executed separately.
- At runtime, threads in a program have a common memory space and can therefore share data and code, i.e. they are *lightweight*.
- They also share the process running the program.
- Java threads make the runtime environment asynchronous, allowing different tasks to be performed concurrently.

Main Thread

- The runtime environment distinguishes between *user threads* and *daemon threads*.
- As long as a user thread is alive, the execution does not terminate.
- A daemon thread is at the mercy of the runtime system: it is stopped if there are no more user threads running, thus terminating the program.
- Daemon threads exist only to serve user threads.
- *Main thread*: When a standalone application is run, a user thread is automatically created to execute the `main()` method.
- If no other threads are spawned, the program terminates when the `main()` method finishes executing.
- All other threads, called *child threads*, are spawned from the main thread, inheriting its user-thread status.
- The `main()` method can then finish, but the program will keep running until all the user threads are done.

- A thread's status (daemon or user) can be changed by using the method `setDaemon(boolean)`, but this must be done before the thread is started.
- Changing the status of a thread already started throws the `IllegalThreadStateException`.
- Marking all threads spawned as daemon threads ensures that the application terminates when the main thread dies.
- When a GUI application is started, an *AWT thread* is automatically created to monitor the user-GUI interaction.

Thread Creation

- A thread in Java is represented by an object of the Thread class.
- Implementing threads is achieved in one of two ways:
 - Implementing the `java.lang.Runnable` Interface
 - Extending the `java.lang.Thread` Class

Implementing the Runnable Interface

```
public interface Runnable {  
    public void run();  
}
```

- The code in the `run()` method defines an independent path of execution and thereby the entry and the exits for the thread.
- The thread ends when the `run()` method ends.
- The procedure for creating threads based on the `Runnable` interface is as follows:
 1. A class implements the `Runnable` interface providing the `run()` method which will be executed by the thread.
 2. An object of `Thread` class is created. An object of a class implementing the `Runnable` interface is passed as an argument to a constructor of the `Thread` class.
 3. The `start()` method is invoked on the `Thread` object created in the previous step. However, the `start()` method returns immediately after a thread has been spawned. This method is defined in the `Thread` class.
- Invocation of the `start()` method on the `Thread` object, which has been passed the object implementing the `Runnable` interface, will eventually result in the `run()` method being executed by the thread represented by the `Thread` object.

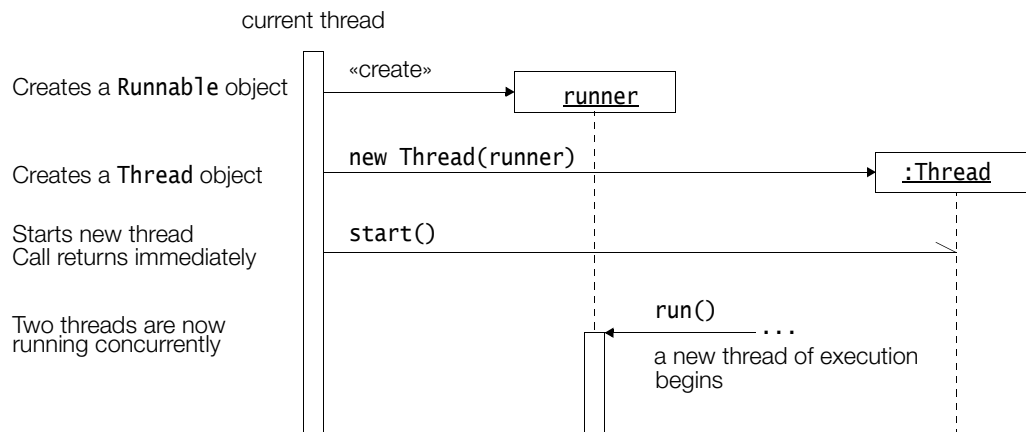


Figure 1 Spawning Threads

Example 1 Implementing the Runnable Interface

```

class Counter implements Runnable {
    private int currentValue;
    private Thread worker;
    public Counter(String threadName) {
        currentValue = 0;
        worker = new Thread(this, threadName); // (1) Create a new thread
        System.out.println(worker);
        worker.start(); // (2) Start the thread
    }
    public void run() { // (3) Thread entry point
        try {
            while (currentValue < 5) {
                System.out.println(worker.getName() + ": " + (currentValue++));
                Thread.sleep(500); // (4) Current thread sleeps
            }
        } catch (InterruptedException e) {
            System.out.println(worker.getName() + " interrupted.");
        }
        System.out.println("Exit from " + worker.getName() + ".");
    }
    public int getValue() { return currentValue; }
}

```

```

public class Client {
    public static void main(String args[]) {
        Counter counterA = new Counter("Counter A"); // (5) Create a thread
        try {
            int val;
            do {
                val = counterA.getValue(); // (6) Access the counter value
                System.out.println("Main Thread: " + val);
                Thread.sleep(1000); // (7) Current thread sleeps
            } while (val < 5);
        } catch (InterruptedException e) {
            System.out.println("Main Thread interrupted.");
        }
        System.out.println("Exit from Main Thread.");
    }
}

```

- Possible output from the program:

```

Thread[Counter A,5,main]
Main Thread: 0
Counter A: 0
Counter A: 1
Main Thread: 2
Counter A: 2
Counter A: 3
Main Thread: 4

```

```

Counter A: 4
Exit from Counter A.
Main Thread: 5
Exit from Main Thread.

```

- Note that the main thread executing in the `Client` class sleeps for a longer time between iterations than the `Counter` thread, giving the `Counter` thread the opportunity to be executed.
- The `Counter` thread is a *child* thread of the main thread.
- It inherits the user-thread status from the main thread.
- If code of the try-catch block in the `main()` method of the `Client` class is removed, the main thread would finish executing before the child thread.
- However, the program would run until the child thread completes.
- Since thread scheduling is not predictable and the example above does not enforce any synchronization between the two threads in accessing the counter value, the output shown may vary.

Extending the Thread Class

1. A class extending the Thread class overrides the run() method from the Thread class to define the code executed by the thread.
 2. This subclass may call a Thread constructor explicitly in its constructors to initialize the thread.
 3. The start() method inherited from the Thread class is invoked on the object of the class to make the thread eligible for running.
- The Thread class implements the Runnable interface.
 - Creation of the Runnable object and the Thread object are combined in the subclass, with the implementation of the run() method in the subclass being invoked when the thread is run.
 - When creating threads, implementing the Runnable interface is usually preferred.

Example 2 Extending the Thread Class

```
class Counter extends Thread {
    private int currentValue;
    public Counter(String threadName) {
        super(threadName);           // (1) Initialize thread
        currentValue = 0;
        System.out.println(this);
        start();                     // (2) Start this thread
    }
    public void run() {              // (3) Override from superclass
        try {
            while (currentValue < 5) {
                System.out.println(getName() + ": " + (currentValue++));
                Thread.sleep(500);    // (4) Current thread sleeps
            }
        } catch (InterruptedException e) {
            System.out.println(getName() + " interrupted.");
        }
        System.out.println("Exit from " + getName() + ".");
    }
    public int getValue() { return currentValue; }
}
```

```
public class Client {  
    public static void main(String args[]) {  
        Counter counterA = new Counter("Counter A"); // (5) Create a thread  
        System.out.println("Exit from Main Thread.");  
    }  
}
```

- Possible output from the program:

```
Thread[Counter A,5,main]  
Exit from Main Thread.  
Counter A: 0  
Counter A: 1  
Counter A: 2  
Counter A: 3  
Counter A: 4  
Exit from Counter A.
```

Synchronization

- Threads share the same memory space, i.e. they can share resources.
- Critical situations may require that only one thread at a time has access to a shared resource.

Monitors

- A *monitor* (a.k.a. *semaphore*) is used to synchronize access to a shared resource.
 - A region of code, representing a shared resource, can be associated with a monitor.
- Threads gain access to a shared resource by first acquiring the monitor associated with the resource.
- At any given time, no more than one thread can own the monitor and thereby have access to the shared resource.
- A monitor thus implements a *mutually exclusive locking mechanism* (a.k.a. *mutex*).
- The monitor mechanism enforces the following rules of synchronization:
 - No other thread can *enter* a monitor if a thread has already acquired the monitor. Threads wishing to acquire the monitor will *wait* for the monitor to become available.
 - When a thread *exits* a monitor, a waiting thread is given the monitor, and can proceed to access the shared resource associated with this monitor.

- All Java objects have a monitor, and each object can be used as a mutually exclusive lock.
- There are two ways in which code can be synchronized:
 - Synchronized Methods
 - Synchronized Blocks

Synchronized Methods

- If the methods of an object should only be executed by one thread at a time, then the definitions of all such methods should be modified with the keyword `synchronized`.
- A thread wishing to execute a synchronized method must first enter the object's monitor (i.e. gain ownership of the monitor).
 - This is simply achieved by calling the method.
 - If the monitor is already owned by another thread, the calling thread waits.
 - A thread exits a monitor simply by returning from the synchronized method, allowing the next waiting thread, if any, to enter.
- Synchronized methods are useful in situations where methods can manipulate the state of an object in ways that can corrupt the state if executed concurrently.

```
class StackImpl {  
    private Object stackArray[];  
    private int topOfStack;  
  
    public synchronized Object pop() {           // synchronized method  
        Object obj = stackArray[topOfStack];  
        stackArray[topOfStack--] = null;  
        return obj;  
    }  
}
```

```
    public synchronized void push(Object element) // synchronized method  
    { stackArray[++topOfStack] = element; }  
    // ...  
}
```

- While a thread is inside a synchronized method of an object, all other threads that wish to execute this synchronized method or any other synchronized method of the object will have to wait.
 - This restriction does not apply to the thread that already has the monitor and is executing a synchronized method of the object.
 - Such a method can invoke other synchronized methods of the object without being blocked.
 - The non-synchronized methods of the object can of course be called at any time by any thread.
- Synchronized methods can also be static.
 - Such a monitor is actually a monitor on the `Class` object of the class.
 - Synchronization of static methods in a class is independent from the synchronization of instance methods on objects of the class.
- A subclass decides whether the new definition of an inherited synchronized method will remain synchronized in the subclass.

Synchronized Blocks

- The synchronized block allows arbitrary code to be synchronized on the monitor of an arbitrary object.

```
synchronized (<object reference>) { <code block> }
```

- The code block is usually related to the object on which the synchronization is being done.
- Once a thread has entered the code block after acquiring the monitor on the specified object, no other thread will be able to execute the code block, or any other code requiring the monitor, until the monitor is released on this object.
- In contrast to synchronized methods, this mechanism allows fine-grained synchronization of code on arbitrary objects.
- Object specification in the synchronized statement is mandatory.

```
class SmartClient {  
    BankAccount account;  
    // ...  
    public void updateTransaction() {  
        synchronized (account) {           // (1) synchronized block  
            account.update();               // (2)  
        }  
    }  
}
```

- Inner classes can access data in their enclosing context. An inner object might need to synchronize on its associated outer object, in order to ensure integrity of data in the latter.
- Synchronizing on an inner object and on its associated outer object are independent of each other, unless enforced explicitly, as in the code below.

```
class Outer_A {                               // (1) Top-level Class  
    private double x;                         // (2)  
    protected class Inner_B {                // (3) Non-static inner Class  
        public void setX() {                 // (4)  
            synchronized(Outer_A.this) {     // (5) Synchronized block  
                x = Math.PI;                 // (6)  
            }  
        }  
    }  
}
```

Thread Transitions

Thread States

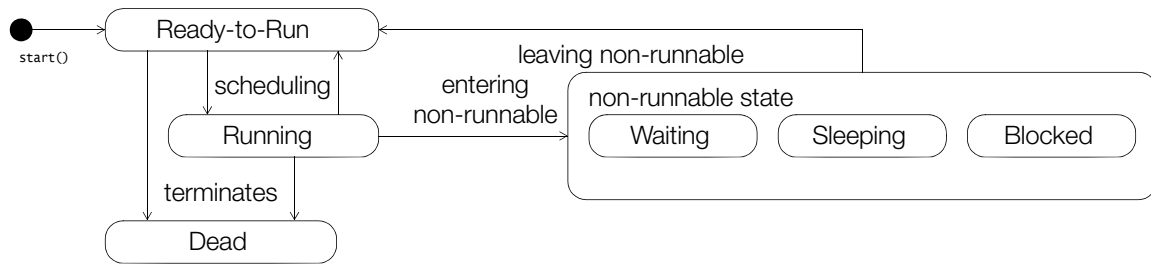


Figure 2 Thread States

- Just because a thread's `start()` method has been called, it does not mean that the thread has access to the CPU and can start executing straight away.

- A thread can be in one of the following states:
 - *Running state*: This means that the CPU is currently executing the thread. The *thread scheduler* decides which thread is in the Running state.
 - *Non-runnable states*: A thread can go from the Running state into one of the non-runnable states, depending on the transition. A thread remains in a non-runnable state until a special transition moves it to the Ready-to-run state.
 - *Waiting state*
 - *Sleeping state*
 - *Blocked state*
 - *Ready-to-run state*:
 - In the Ready-to-run state, a thread awaits its turn to get CPU time. The thread scheduler decides which thread gets to run.
 - *Dead state*:
 - The thread can be dead because it has completed or it has been terminated.
 - Once in this state, the thread cannot be resurrected.

Thread Priorities

- Threads are assigned priorities that the thread scheduler can use to determine how the threads will be treated.
- Priorities are integer values from 1 (lowest priority given by the constant `Thread.MIN_PRIORITY`) to 10 (highest priority given by the constant `Thread.MAX_PRIORITY`).
 - Default priority of 5 (`Thread.NORM_PRIORITY`).
- A thread inherits the priority of its parent thread.

```
myThread.setPriority(Math.min(Thread.MAX_PRIORITY, myThread.getPriority()+1));
```

Thread Scheduler

- Schedulers usually employ one of the two following strategies:
 - Preemptive Scheduling.
If a thread with a higher priority than the current running thread moves to the Ready-to-run state, then the current running thread can be *preempted* (moved to the Ready-to-run state) to let the higher priority thread execute.
 - Time-sliced or Round-robin Scheduling.
A running thread is allowed to execute for a fixed length of time, after which it moves to the Ready-to-run state to await its turn to run again.
- Thread schedulers are implementation- and platform-dependent.

Running and Yielding

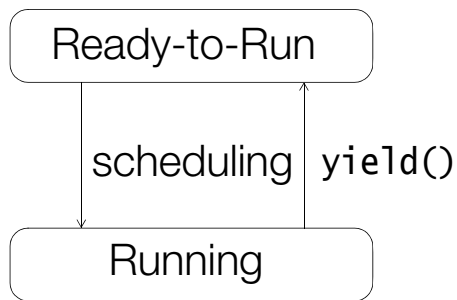


Figure 3 Running and Yielding

```
public void run() {
    try {
        while (!done()) {
            doLittleBitMore();
            Thread.yield();           // current thread yields
        }
    } catch (InterruptedException e) {
        doCleaningUp();
    }
}
```

Sleeping and Waking Up

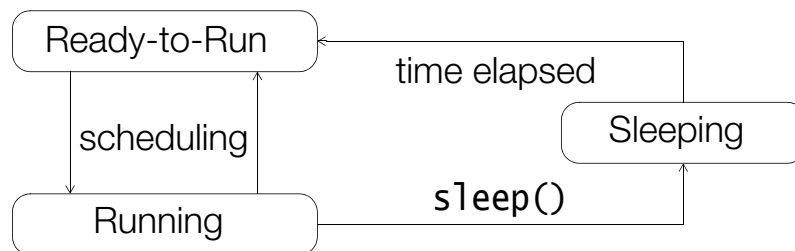


Figure 4 Sleeping and Waking up

Waiting and Notifying

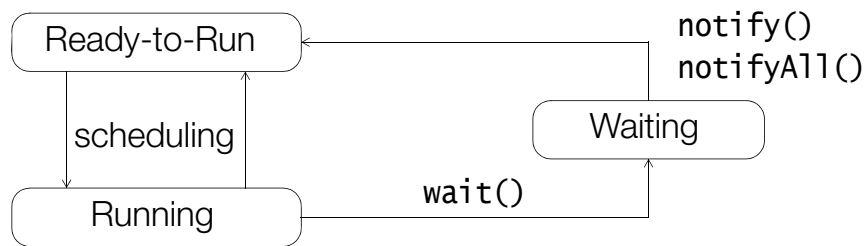


Figure 5 Waiting and Notifying

- Waiting and notifying provide a means of communication between threads that *synchronize on the same object*.
- The following methods in the `Object` class can be used for this purpose:

```
void wait(long timeout) throws InterruptedException
void wait(long timeout, int nanos) throws InterruptedException
void wait() throws InterruptedException
void notify()
void notifyAll()
```

- The `wait()`, `notify()` and `notifyAll()` methods must be executed in synchronized code, otherwise the call will result in an `IllegalMonitorStateException`.
- A thread waiting as a result of calling `wait()` on an object must be notified by another thread calling `notify()` on the same object, in order for it to start running again.
- Threads that start running again as a result of being notified will regain the monitor of the object. A call to `notify()` has no consequences if there are no threads waiting.
- A call to the `notifyAll()` method can be used to enable all such threads for running.

Example 3 *Waiting and Notifying*

```
class StackImpl {
    private Object stackArray[];
    private volatile int topOfStack;

    public StackImpl (int capacity) {
        stackArray = new Object[capacity];
        topOfStack = -1;
    }

    public synchronized Object pop() {
        System.out.println(Thread.currentThread() + ": popping");
        while (isEmpty())
            try {
                System.out.println(Thread.currentThread() + ": waiting to pop");
                wait(); // (1)
            } catch (InterruptedException e) { }
        Object obj = stackArray[topOfStack];
        stackArray[topOfStack--] = null;
        System.out.println(Thread.currentThread() + ": notifying after pop");
        notify(); // (2)
        return obj;
    }
}
```

```
    public synchronized void push(Object element) {
        System.out.println(Thread.currentThread() + ": pushing");
        while (isFull())
            try {
                System.out.println(Thread.currentThread() + ": waiting to push");
                wait(); // (3)
            } catch (InterruptedException e) { }
        stackArray[++topOfStack] = element;
        System.out.println(Thread.currentThread() + ": notifying after push");
        notify(); // (4)
    }

    public boolean isFull() { return topOfStack == stackArray.length -1; }
    public boolean isEmpty() { return topOfStack < 0; }
}

abstract class StackUser extends Thread { // (5) Stack user
    protected StackImpl stack; // (6)

    public StackUser(String threadName, StackImpl aStack) {
        super(threadName);
        stack = aStack;
        System.out.println(this);
        setDaemon(true); // (7) Daemon thread
        start(); // (8) Start this thread
    }
}
```



```

class StackPopper extends StackUser {                // (9) Popper
    public StackPopper(String threadName, StackImpl aStack) {
        super(threadName, aStack);
    }
    public void run() { while (true) stack.pop(); }
}

class StackPusher extends StackUser {                // (10) Pusher
    public StackPusher(String threadName, StackImpl aStack) {
        super(threadName, aStack);
    }
    public void run() { while (true) stack.push(new Integer(1)); }
}

public class WaitAndNotifyTest {
    public static void main(String args[])
        throws InterruptedException {                // (11)

        StackImpl stack = new StackImpl(5);
        new StackPusher("A", stack);
        new StackPusher("B", stack);
        new StackPopper("C", stack);
        System.out.println("Main Thread sleeping.");
        Thread.sleep(5000);
        System.out.println("Exit from Main Thread.");
    }
}

```

- Possible output from the program:

```

Thread[A,5,main]
Thread[B,5,main]
Thread[C,5,main]
Main Thread sleeping.
...
Thread[A,5,main]: pushing
Thread[A,5,main]: waiting to push
Thread[B,5,main]: pushing
Thread[B,5,main]: waiting to push
Thread[C,5,main]: popping
Thread[C,5,main]: notifying after pop
Thread[A,5,main]: notifying after push
Thread[A,5,main]: pushing
Thread[A,5,main]: waiting to push
Thread[B,5,main]: waiting to push
Thread[C,5,main]: popping
Thread[C,5,main]: notifying after pop
Thread[A,5,main]: notifying after push
...
Thread[B,5,main]: notifying after push
...
Exit from Main Thread.
...

```

- In Example 3, three daemon threads are manipulating the same stack.
 - Two of them are pushing an element on the stack, while the third one is popping the stack.
- The instance variable `topOfStack` in class `StackImpl` is declared `volatile`, so that read and write operations on this variable will access the “master” value of this variable, and not any copies, during runtime.
- Both synchronized methods `pop()` and `push()` have calls to `wait()` and `notify()` methods.
- When a thread executing the synchronized method `push()` on an object of class `StackImpl` pushes an element on the stack, it calls the `notify()` method at (4).
- Each push and pop operation can be traced by a sequence consisting of the name of the operation to be performed, followed by zero or more wait messages, and concluding with a notification after the operation is done.

```
Thread[A,5,main]: pushing
Thread[A,5,main]: waiting to push
...
Thread[A,5,main]: notifying after push
Thread[A,5,main]: pushing
Thread[A,5,main]: waiting to push
...
Thread[A,5,main]: notifying after push
```

- Thread B is shown doing one push:

```
Thread[B,5,main]: pushing
Thread[B,5,main]: waiting to push
...
Thread[B,5,main]: notifying after push
```

- Whereas thread C pops the stack twice without any waiting:

```
Thread[C,5,main]: popping
Thread[C,5,main]: notifying after pop
...
Thread[C,5,main]: popping
Thread[C,5,main]: notifying after pop
```

- When the operations are interweaved, the output clearly shows that the pushers wait when the stack is full, and only push after the stack is popped.

Miscellaneous Methods in the Thread Class

`boolean isAlive()`

A thread being alive means that it has been started but has not terminated.

`void join() throws InterruptedException`

A call to this method invoked on a thread will wait and not return until the thread has completed.

A parent thread can use this method to wait for its child thread to complete before continuing, i.e. the parent thread waits for the child thread to *join it after completion*.

Example 4 Usage of `isAlive()` and `join()` Methods

```
class Counter extends Thread { /* See Example 2. */ }
public class AnotherClient {
    public static void main(String args[]) {
        Counter cA = new Counter("Counter A");
        Counter cB = new Counter("Counter B");
        try {
            System.out.println("Wait for the child threads to finish.");
            cA.join(); // (5)
            cB.join(); // (6)
            if (!cA.isAlive()) // (7)
                System.out.println("Counter A not alive.");
            if (!cB.isAlive()) // (8)
                System.out.println("Counter B not alive.");
        } catch (InterruptedException e) {
            System.out.println("Main Thread interrupted.");
        }
        System.out.println("Exit from Main Thread.");
    }
}
```

- Possible output from the program:

```
Thread[Counter A,5,main]
Thread[Counter B,5,main]
Wait for the child threads to finish.
Counter A: 0
Counter B: 0
Counter A: 1
Counter B: 1
Counter A: 2
Counter B: 2
Counter A: 3
Counter B: 3
Counter A: 4
Counter B: 4
Exit from Counter A.
Exit from Counter B.
Counter A not alive.
Counter B not alive.
Exit from Main Thread.
```